Accounting Module - Manual Testing Guide

> **Base URL**: `http://localhost:3000/api/accounting`
> **Auth**: All requests require the user to be logged in with an active session (cookie-based). The user must have one of the accounting roles: `finance_manager`, `accountant`, or `finance_viewer`.

Prerequisites

Before testing, ensure:

1. The app is running locally (`npm run dev`)
2. You are logged in as a user with `finance_manager` role (has full access)
3. The `accounting` feature flag is enabled for your organization
4. You have at least one organization with members

Phase 1: Core General Ledger - Legal Entities & COA

Problem 1.1: Initialize Accounting Defaults

The system needs default accounts, fiscal calendar, and settings before any accounting work can begin.

How to test:

```
curl -X POST http://localhost:3000/api/accounting/init-defaults \
  -H "Content-Type: application/json" \
  -H "Cookie: <your-session-cookie>" \
  -d '{}'
```

Expected output:

```
{
  "data": {
    "id": "<uuid>",
    "organization_id": "<your-org-id>",
    "default_currency": "SAR",
    "fiscal_year_start_month": 1,
    "created_at": "2026-07-23T..."
  }
}
```

This creates:

- A default legal entity (named after your organization)
- Default chart of accounts (assets, liabilities, equity, revenue, expenses)
- A fiscal calendar starting January
- Accounting settings record

Problem 1.2: Create a Legal Entity

Organizations can have multiple legal entities (e.g., a holding company with subsidiaries).

How to test:

```
curl -X POST http://localhost:3000/api/accounting/entities \
  -H "Content-Type: application/json" \
  -H "Cookie: <your-session-cookie>" \
  -d '{
    "name_ar": "شركة التقنية المتقدمة",
    "name_en": "Advanced Tech Company",
    "country": "SA",
    "functional_currency_code": "SAR",
    "tax_id": "300000000000003",
    "commercial_registration": "1010999999"
  }'
```

Expected output:

```
{
  "data": {
    "id": "<uuid>",
    "organization_id": "<org-id>",
    "name_ar": "شركة التقنية المتقدمة",
    "name_en": "Advanced Tech Company",
    "country": "SA",
    "functional_currency_code": "SAR",
    "tax_id": "300000000000003",
    "status": "active",
    "is_default": false,
    "version": 1
  }
}
```


Problem 1.3: Create a Branch Under an Entity

Branches represent physical locations or departments under a legal entity.

How to test:

```
curl -X POST http://localhost:3000/api/accounting/branches \
  -H "Content-Type: application/json" \
  -H "Cookie: <your-session-cookie>" \
  -d '{
    "legal_entity_id": "<entity-id-from-1.2>",
    "name_ar": "فرع الرياض",
    "name_en": "Riyadh Branch",
    "code": "RYD-01"
  }'
```

Expected output:

```
{
  "data": {
    "id": "<uuid>",
    "organization_id": "<org-id>",
    "legal_entity_id": "<entity-id>",
    "name_ar": "فرع الرياض",
    "name_en": "Riyadh Branch",
    "code": "RYD-01",
    "is_active": true,
    "version": 1
  }
}
```

Problem 1.4: Create a Fiscal Calendar

Fiscal calendars define the financial year periods.

How to test:

```
curl -X POST http://localhost:3000/api/accounting/fiscal-calendars \
  -H "Content-Type: application/json" \
  -H "Cookie: <your-session-cookie>" \
  -d '{
    "name_ar": "السنة المالية 2026",
    "name_en": "Fiscal Year 2026",
    "year": 2026,
    "start_month": 1
  }'
```

Expected output:

```
{
  "data": {
    "id": "<uuid>",
    "organization_id": "<org-id>",
    "name_ar": "2026 السنة المالية",
    "name_en": "Fiscal Year 2026",
    "year": 2026,
    "start_month": 1,
    "is_active": true
  }
}
```


Problem 1.5: List the Chart of Accounts

View all accounts created by the initialization.

How to test:

```
curl http://localhost:3000/api/accounting/accounts \
  -H "Cookie: <your-session-cookie>"
```

Expected output:

```
{
  "data": [
    {
      "id": "<uuid>",
      "code": "1000",
      "name_ar": "الأصول",
      "name_en": "Assets",
      "type": "asset",
      "is_group": true,
      "parent_id": null
    },
    {
      "id": "<uuid>",
      "code": "1100",
      "name_ar": "النقد وما في حكمه",
      "name_en": "Cash and Cash Equivalents",
      "type": "asset",
      "parent_id": "<assets-group-id>"
    }
    // ... more accounts
  ]
}
```

Phase 2: Dimensions, Cost Centers & Settings

Problem 2.1: Create a Cost Center

Cost centers track expenses by department/project.

How to test:

```
curl -X POST http://localhost:3000/api/accounting/cost-centers \
  -H "Content-Type: application/json" \
  -H "Cookie: <your-session-cookie>" \
  -d '{
    "code": "IT-001",
    "name_ar": "قسم تقنية المعلومات",
    "name_en": "IT Department",
    "is_active": true
  }'
```

Expected output:

```
{
  "data": {
    "id": "<uuid>",
    "organization_id": "<org-id>",
    "code": "IT-001",
    "name_ar": "قسم تقنية المعلومات",
    "name_en": "IT Department",
    "is_active": true
  }
}
```

Problem 2.2: Create a Dimension Type

Dimensions allow multi-axis reporting (e.g., project, region, product line).

How to test:

```
curl -X POST http://localhost:3000/api/accounting/dimension-types \
  -H "Content-Type: application/json" \
  -H "Cookie: <your-session-cookie>" \
  -d '{
    "code": "REGION",
    "name_ar": "المنطقة",
    "name_en": "Region",
    "is_active": true
  }'
```

Expected output:

```
{
  "data": {
    "id": "<uuid>",
    "organization_id": "<org-id>",
    "code": "REGION",
    "name_ar": "المنطقة",
    "name_en": "Region",
    "is_active": true
  }
}
```

Problem 2.3: Create Dimension Values

Values within a dimension type (e.g., "Central", "Western" for region).

How to test:

```
curl -X POST http://localhost:3000/api/accounting/dimension-values \
  -H "Content-Type: application/json" \
  -H "Cookie: <your-session-cookie>" \
  -d '{
    "dimension_type_id": "<dimension-type-id-from-2.2>",
    "code": "CENTRAL",
    "name_ar": "المنطقة الوسطى",
    "name_en": "Central Region",
    "is_active": true
  }'
```

Expected output:

```
{
  "data": {
    "id": "<uuid>",
    "dimension_type_id": "<dimension-type-id>",
    "code": "CENTRAL",
    "name_ar": "المنطقة الوسطى",
    "name_en": "Central Region",
    "is_active": true
  }
}
```

Problem 2.4: Update Accounting Settings

Configure default accounts for automatic posting.

How to test:

```
curl -X POST http://localhost:3000/api/accounting/settings \
  -H "Content-Type: application/json" \
  -H "Cookie: <your-session-cookie>" \
  -d '{
    "default_currency": "SAR",
    "fiscal_year_start_month": 1,
    "vat_rate": 15,
    "auto_post_payroll": true,
    "default_receivable_account_id": "<receivable-account-uuid>",
    "default_payable_account_id": "<payable-account-uuid>"
  }'
```

Expected output:

```
{
  "data": {
    "id": "<uuid>",
    "organization_id": "<org-id>",
    "default_currency": "SAR",
    "vat_rate": 15,
    "auto_post_payroll": true,
    "fiscal_year_start_month": 1
  }
}
```


Phase 3: Journal Entries & Workflows

Problem 3.1: Create a Manual Journal Entry

Journal entries are the fundamental accounting record (debit = credit).

How to test:

```
curl -X POST http://localhost:3000/api/accounting/journals \
  -H "Content-Type: application/json" \
  -H "Cookie: <your-session-cookie>" \
  -d '{
    "journal_date": "2026-07-01",
    "description": "Office rent payment",
    "lines": [
      {
        "account_id": "<rent-expense-account-id>",
        "debit": 5000,
        "credit": 0,
        "description": "Monthly rent"
      },
      {
        "account_id": "<cash-account-id>",
        "debit": 0,
        "credit": 5000,
        "description": "Cash payment"
      }
    ]
  }'
```

Expected output:

```
{
  "data": {
    "id": "<uuid>",
    "entry_no": "JV-000001",
    "organization_id": "<org-id>",
    "journal_date": "2026-07-01",
    "description": "Office rent payment",
    "status": "draft",
    "total_debit": 5000,
    "total_credit": 5000,
    "lines": [
      {"account_id": "...", "debit": 5000, "credit": 0},
      {"account_id": "...", "debit": 0, "credit": 5000}
    ]
  }
}
```

Problem 3.2: Post a Journal Entry

Posting makes the journal entry final and updates account balances.

How to test:

```
curl -X POST http://localhost:3000/api/accounting/journals/<journal-id>/post \
  -H "Content-Type: application/json" \
  -H "Cookie: <your-session-cookie>" \
  -d '{}'
```

Expected output:

```
{
  "data": {
    "id": "<journal-id>",
    "status": "posted",
    "posted_at": "2026-07-23T...",
    "posted_by": "<user-id>"
  }
}
```

Problem 3.3: Reverse a Posted Journal Entry

Reversals create a mirror-image entry to cancel a posted journal.

How to test:

```
curl -X POST http://localhost:3000/api/accounting/journals/<journal-id>/reverse \
  -H "Content-Type: application/json" \
  -H "Cookie: <your-session-cookie>" \
  -d '{
    "reason": "Incorrect amount recorded"
  }'
```

Expected output:

```
{
  "data": {
    "id": "<new-reversal-journal-id>",
    "entry_no": "JV-000002",
    "status": "posted",
    "description": "Reversal: Office rent payment",
    "reversal_of": "<original-journal-id>"
  }
}
```

Problem 3.4: Create an Approval Policy

Approval policies define who must approve journal entries above certain thresholds.

How to test:

```
curl -X POST http://localhost:3000/api/accounting/approval-policies \
  -H "Content-Type: application/json" \
  -H "Cookie: <your-session-cookie>" \
  -d '{
    "name_ar": "سياسة اعتماد القيود فوق 10000",
    "name_en": "Journals above 10000 approval",
    "document_type": "journal_entry",
    "min_amount": 10000,
    "is_active": true
  }'
```

Expected output:

```
{
  "data": {
    "id": "<uuid>",
    "organization_id": "<org-id>",
    "name_ar": "سياسة اعتماد القيود فوق 10000",
    "name_en": "Journals above 10000 approval",
    "document_type": "journal_entry",
    "min_amount": 10000,
    "is_active": true
  }
}
```


Problem 3.5: Create a Recurring Journal Template

Templates for entries that repeat monthly (e.g., rent, depreciation).

How to test:

```
curl -X POST http://localhost:3000/api/accounting/recurring-templates \
  -H "Content-Type: application/json" \
  -H "Cookie: <your-session-cookie>" \
  -d '{
    "name_ar": "إيجار شهري",
    "name_en": "Monthly Rent",
    "description": "Recurring office rent",
    "frequency": "monthly",
    "is_active": true,
    "lines": [
      {"account_id": "<rent-expense-id>", "debit": 5000, "credit": 0},
      {"account_id": "<cash-id>", "debit": 0, "credit": 5000}
    ]
  }'
```

Expected output:

```
{
  "data": {
    "id": "<uuid>",
    "name_ar": "إيجار شهري",
    "name_en": "Monthly Rent",
    "frequency": "monthly",
    "is_active": true
  }
}
```

Problem 3.6: Create a Period Close Checklist

Checklists ensure all month-end tasks are completed before closing.

How to test:

```
curl -X POST http://localhost:3000/api/accounting/close-checklist-templates \
  -H "Content-Type: application/json" \
  -H "Cookie: <your-session-cookie>" \
  -d '{
    "name_ar": "قائمة إغلاق نهاية الشهر",
    "name_en": "Month-End Close Checklist",
    "period_type": "monthly",
    "is_active": true
  }'
```

Expected output:

```
{
  "data": {
    "id": "<uuid>",
    "name_ar": "قائمة إغلاق نهاية الشهر",
    "name_en": "Month-End Close Checklist",
    "period_type": "monthly",
    "is_active": true
  }
}
```

Phase 4: Treasury & Bank Reconciliation

Problem 4.1: Create a Bank Account

Register company bank accounts for reconciliation.

How to test:

```
curl -X POST http://localhost:3000/api/accounting/bank-accounts \
  -H "Content-Type: application/json" \
  -H "Cookie: <your-session-cookie>" \
  -d '{
    "name_ar": "الحساب الجاري - البنك الأهلي",
    "name_en": "Current Account - Al Ahli Bank",
    "bank_name": "البنك الأهلي السعودي",
    "account_number": "SA0380000000608010167519",
    "currency_code": "SAR",
    "gl_account_id": "<bank-gl-account-id>",
    "is_active": true
  }'
```

Expected output:

```
{
  "data": {
    "id": "<uuid>",
    "organization_id": "<org-id>",
    "name_ar": "الحساب الجاري - البنك الأهلي",
    "name_en": "Current Account - Al Ahli Bank",
    "bank_name": "البنك الأهلي السعودي",
    "account_number": "SA0380000000608010167519",
    "currency_code": "SAR",
    "is_active": true
  }
}
```

Problem 4.2: Upload a Bank Statement

Import statement data for reconciliation.

How to test:

```
curl -X POST http://localhost:3000/api/accounting/bank-statements \
  -H "Content-Type: application/json" \
  -H "Cookie: <your-session-cookie>" \
  -d '{
    "bank_account_id": "<bank-account-id-from-4.1>",
    "statement_date": "2026-07-15",
    "opening_balance": 50000,
    "closing_balance": 45000,
    "currency_code": "SAR"
  }'
```

Expected output:

```
{
  "data": {
    "id": "<uuid>",
    "bank_account_id": "<bank-account-id>",
    "statement_date": "2026-07-15",
    "opening_balance": 50000,
    "closing_balance": 45000,
    "status": "draft"
  }
}
```

Problem 4.3: Create a Bank Reconciliation

Match statement lines against GL entries.

How to test:

```
curl -X POST http://localhost:3000/api/accounting/bank-reconciliations \
  -H "Content-Type: application/json" \
  -H "Cookie: <your-session-cookie>" \
  -d '{
    "bank_account_id": "<bank-account-id>",
    "period_start": "2026-07-01",
    "period_end": "2026-07-31",
    "statement_balance": 45000,
    "gl_balance": 45500
  }'
```

Expected output:


```
{
  "data": {
    "id": "<uuid>",
    "bank_account_id": "<bank-account-id>",
    "period_start": "2026-07-01",
    "period_end": "2026-07-31",
    "statement_balance": 45000,
    "gl_balance": 45500,
    "difference": -500,
    "status": "draft"
  }
}
```


Problem 4.4: Record a Cash Position

Track real-time cash positions across accounts.

How to test:

```
curl -X POST http://localhost:3000/api/accounting/cash-positions \
  -H "Content-Type: application/json" \
  -H "Cookie: <your-session-cookie>" \
  -d '{
    "bank_account_id": "<bank-account-id>",
    "position_date": "2026-07-23",
    "balance": 45000,
    "available_balance": 44000,
    "currency_code": "SAR"
  }'
```

Expected output:

```
{
  "data": {
    "id": "<uuid>",
    "bank_account_id": "<bank-account-id>",
    "position_date": "2026-07-23",
    "balance": 45000,
    "available_balance": 44000
  }
}
```

Problem 4.5: Create a Payment Batch

Batch multiple payments for bank processing.

How to test:

```
curl -X POST http://localhost:3000/api/accounting/payment-batches \
  -H "Content-Type: application/json" \
  -H "Cookie: <your-session-cookie>" \
  -d '{
    "name_ar": "دفعة رواتب يوليو",
    "name_en": "July Salary Batch",
    "bank_account_id": "<bank-account-id>",
    "payment_date": "2026-07-25",
    "total_amount": 150000
  }'
```

Expected output:

```
{
  "data": {
    "id": "<uuid>",
    "name_ar": "دفعة رواتب يوليو",
    "name_en": "July Salary Batch",
    "payment_date": "2026-07-25",
    "total_amount": 150000,
    "status": "draft"
  }
}
```

Phase 5: Fixed Assets

Problem 5.1: Create an Asset Category

Categories define GL account mappings and depreciation defaults.

How to test:

```
curl -X POST http://localhost:3000/api/accounting/asset-categories \
  -H "Content-Type: application/json" \
  -H "Cookie: <your-session-cookie>" \
  -d '{
    "name_ar": "أجهزة حاسب آلي",
    "name_en": "Computer Equipment",
    "code": "IT-EQUIP",
    "useful_life_months": 36,
    "depreciation_method": "straight_line",
    "asset_account_id": "<fixed-assets-gl-account-id>",
    "depreciation_account_id": "<depreciation-expense-gl-account-id>",
    "accumulated_depreciation_account_id": "<accum-depreciation-gl-account-id>"
  }'
```

Expected output:

```
{
  "data": {
    "id": "<uuid>",
    "code": "IT-EQUIP",
    "name_ar": "أجهزة حاسب آلي",
    "name_en": "Computer Equipment",
    "useful_life_months": 36,
    "depreciation_method": "straight_line"
  }
}
```

Problem 5.2: Register a Fixed Asset

Record a new asset acquisition.

How to test:

```
curl -X POST http://localhost:3000/api/accounting/fixed-assets \
  -H "Content-Type: application/json" \
  -H "Cookie: <your-session-cookie>" \
  -d '{
    "category_id": "<category-id-from-5.1>",
    "name_ar": "خادم ديل R740",
    "name_en": "Dell Server R740",
    "asset_code": "FA-IT-001",
    "acquisition_date": "2026-07-01",
    "acquisition_cost": 25000,
    "useful_life_months": 36,
    "depreciation_method": "straight_line",
    "salvage_value": 2500
  }'
```

Expected output:

```
{
  "data": {
    "id": "<uuid>",
    "asset_code": "FA-IT-001",
    "name_ar": "خادم ديل R740",
    "name_en": "Dell Server R740",
    "acquisition_cost": 25000,
    "useful_life_months": 36,
    "salvage_value": 2500,
    "status": "draft",
    "net_book_value": 25000
  }
}
```


Problem 5.3: Run Monthly Depreciation

Process depreciation for all active assets.

How to test:

```
curl -X POST http://localhost:3000/api/accounting/depreciation-runs \
  -H "Content-Type: application/json" \
  -H "Cookie: <your-session-cookie>" \
  -d '{
    "run_date": "2026-07-31",
    "period": "2026-07"
  }'
```

Expected output:


```
{
  "data": {
    "id": "<uuid>",
    "run_date": "2026-07-31",
    "period": "2026-07",
    "status": "draft",
    "total_depreciation": 625.00
  }
}
```

The monthly depreciation for a 25,000 SAR asset with 2,500 salvage over 36 months = (25000 - 2500) / 36 = 625 SAR/month.

Problem 5.4: Record an Asset Revaluation

Adjust asset value due to market changes or impairment.

How to test:

```
curl -X POST http://localhost:3000/api/accounting/fixed-assets/<asset-id>/revalue \
  -H "Content-Type: application/json" \
  -H "Cookie: <your-session-cookie>" \
  -d '{
    "revaluation_date": "2026-07-23",
    "new_value": 22000,
    "reason": "Market value assessment"
  }'
```

Expected output:

```
{
  "data": {
    "id": "<uuid>",
    "asset_id": "<asset-id>",
    "revaluation_date": "2026-07-23",
    "old_value": 25000,
    "new_value": 22000,
    "difference": -3000
  }
}
```

Phase 6: AR/AP & Party Management

Problem 6.1: Create a Party (Customer/Vendor)

Parties are the unified model for all business relationships.

How to test:

```
curl -X POST http://localhost:3000/api/accounting/parties \
  -H "Content-Type: application/json" \
  -H "Cookie: <your-session-cookie>" \
  -d '{
    "name_ar": "شركة الحلول الذكية",
    "name_en": "Smart Solutions Co.",
    "party_type": "customer",
    "vat_number": "300000000000005",
    "email": "accounts@smartsolutions.sa",
    "phone": "+966501234567",
    "payment_terms_days": 30,
    "credit_limit": 100000,
    "is_active": true
  }'
```

Expected output:

```
{
  "data": {
    "id": "<uuid>",
    "organization_id": "<org-id>",
    "name_ar": "شركة الحلول الذكية",
    "name_en": "Smart Solutions Co.",
    "party_type": "customer",
    "vat_number": "300000000000005",
    "payment_terms_days": 30,
    "credit_limit": 100000,
    "is_active": true,
    "balance": 0
  }
}
```

Problem 6.2: Issue a Credit Note

Credit notes reduce a customer's outstanding balance.

How to test:

```
curl -X POST http://localhost:3000/api/accounting/credit-notes \
  -H "Content-Type: application/json" \
  -H "Cookie: <your-session-cookie>" \
  -d '{
    "party_id": "<party-id-from-6.1>",
    "credit_note_date": "2026-07-20",
    "amount": 2300,
    "reason": "Returned goods - damaged in transit",
    "reference_invoice_id": null
  }'
```

Expected output:

```
{
  "data": {
    "id": "<uuid>",
    "party_id": "<party-id>",
    "credit_note_date": "2026-07-20",
    "amount": 2300,
    "reason": "Returned goods - damaged in transit",
    "status": "draft"
  }
}
```

Problem 6.3: Create a Dunning Policy

Automated reminders for overdue payments.

How to test:

```
curl -X POST http://localhost:3000/api/accounting/dunning-policies \
  -H "Content-Type: application/json" \
  -H "Cookie: <your-session-cookie>" \
  -d '{
    "name_ar": "سياسة التحصيل القياسية",
    "name_en": "Standard Collection Policy",
    "days_overdue_trigger": 30,
    "max_reminders": 3,
    "reminder_interval_days": 7,
    "is_active": true
  }'
```

Expected output:

```
{
  "data": {
    "id": "<uuid>",
    "name_ar": "سياسة التحصيل القياسية",
    "name_en": "Standard Collection Policy",
    "days_overdue_trigger": 30,
    "max_reminders": 3,
    "is_active": true
  }
}
```

Problem 6.4: Generate an Aging Snapshot

Point-in-time report of outstanding balances by age bucket.

How to test:

```
curl -X POST http://localhost:3000/api/accounting/aging-snapshots \
  -H "Content-Type: application/json" \
  -H "Cookie: <your-session-cookie>" \
  -d '{
    "snapshot_date": "2026-07-23",
    "aging_type": "receivable",
    "buckets": [30, 60, 90, 120]
  }'
```

Expected output:

```
{
  "data": {
    "id": "<uuid>",
    "snapshot_date": "2026-07-23",
    "aging_type": "receivable",
    "buckets": [30, 60, 90, 120],
    "total_outstanding": 0
  }
}
```

Phase 7: Budgets & Project Accounting

Problem 7.1: Create a Budget

Budgets set spending limits per account/period.

How to test:

```
curl -X POST http://localhost:3000/api/accounting/budgets \
  -H "Content-Type: application/json" \
  -H "Cookie: <your-session-cookie>" \
  -d '{
    "name": "IT Department Budget 2026",
    "fiscal_year": 2026,
    "lines": [
      {"account_id": "<it-expense-account-id>", "month": 7, "amount": 50000},
      {"account_id": "<it-expense-account-id>", "month": 8, "amount": 50000},
      {"account_id": "<it-expense-account-id>", "month": 9, "amount": 50000}
    ]
  }'
```

Expected output:

```
{
  "data": {
    "id": "<uuid>",
    "name": "IT Department Budget 2026",
    "fiscal_year": 2026,
    "status": "draft",
    "total_amount": 150000
  }
}
```

Problem 7.2: Create a Project

Projects track costs and revenue for specific initiatives.

How to test:

```
curl -X POST http://localhost:3000/api/accounting/projects \
  -H "Content-Type: application/json" \
  -H "Cookie: <your-session-cookie>" \
  -d '{
    "name_ar": "مشروع تطوير البوابة الإلكترونية",
    "name_en": "E-Portal Development Project",
    "code": "PRJ-2026-001",
    "start_date": "2026-07-01",
    "end_date": "2026-12-31",
    "budget_amount": 500000,
    "status": "active"
  }'
```

Expected output:

```
{
  "data": {
    "id": "<uuid>",
    "code": "PRJ-2026-001",
    "name_ar": "مشروع تطوير البوابة الإلكترونية",
    "name_en": "E-Portal Development Project",
    "budget_amount": 500000,
    "actual_cost": 0,
    "status": "active"
  }
}
```


Problem 7.3: Create a Commitment (Purchase Order)

Commitments reserve budget before actual expenditure.

How to test:

```
curl -X POST http://localhost:3000/api/accounting/commitments \
  -H "Content-Type: application/json" \
  -H "Cookie: <your-session-cookie>" \
  -d '{
    "project_id": "<project-id-from-7.2>",
    "description": "Development contractor services",
    "commitment_date": "2026-07-15",
    "amount": 120000,
    "vendor_name": "Tech Contractors LLC"
  }'
```

Expected output:

```
{
  "data": {
    "id": "<uuid>",
    "project_id": "<project-id>",
    "description": "Development contractor services",
    "amount": 120000,
    "consumed_amount": 0,
    "remaining_amount": 120000,
    "status": "open"
  }
}
```

Problem 7.4: Create a Budget Control Rule

Rules that prevent overspending (hard block or warning).

How to test:

```
curl -X POST http://localhost:3000/api/accounting/budget-control-rules \
  -H "Content-Type: application/json" \
  -H "Cookie: <your-session-cookie>" \
  -d '{
    "name_ar": "منع تجاوز الميزانية",
    "name_en": "Block Over-Budget",
    "control_type": "hard_block",
    "threshold_percentage": 100,
    "applies_to": "all_accounts",
    "is_active": true
  }'
```

Expected output:

```
{
  "data": {
    "id": "<uuid>",
    "name_en": "Block Over-Budget",
    "control_type": "hard_block",
    "threshold_percentage": 100,
    "is_active": true
  }
}
```

Phase 8: VAT & ZATCA E-Invoicing

Problem 8.1: Create a Tax Code

Define VAT rates and their GL account mappings.

How to test:

```
curl -X POST http://localhost:3000/api/accounting/tax-codes \
  -H "Content-Type: application/json" \
  -H "Cookie: <your-session-cookie>" \
  -d '{
    "code": "SR",
    "name_ar": "ضريبة القيمة المضافة - المعدل القياسي",
    "name_en": "VAT - Standard Rate",
    "rate": 15,
    "tax_type": "vat",
    "is_active": true
  }'
```

Expected output:

```
{
  "data": {
    "id": "<uuid>",
    "code": "SR",
    "name_ar": "ضريبة القيمة المضافة - المعدل القياسي",
    "name_en": "VAT - Standard Rate",
    "rate": 15,
    "tax_type": "vat",
    "is_active": true
  }
}
```


Problem 8.2: File a VAT Return

Create a quarterly VAT return for ZATCA submission.

How to test:

```
curl -X POST http://localhost:3000/api/accounting/vat-returns \
  -H "Content-Type: application/json" \
  -H "Cookie: <your-session-cookie>" \
  -d '{
    "period_start": "2026-04-01",
    "period_end": "2026-06-30",
    "output_vat": 75000,
    "input_vat": 45000,
    "net_vat": 30000,
    "status": "draft"
  }'
```

Expected output:

```
{
  "data": {
    "id": "<uuid>",
    "period_start": "2026-04-01",
    "period_end": "2026-06-30",
    "output_vat": 75000,
    "input_vat": 45000,
    "net_vat": 30000,
    "status": "draft"
  }
}
```

Problem 8.3: Register a ZATCA Device

Register your e-invoicing compliance device with ZATCA.

How to test:

```
curl -X POST http://localhost:3000/api/accounting/zatca-devices \
  -H "Content-Type: application/json" \
  -H "Cookie: <your-session-cookie>" \
  -d '{
    "device_name": "POS Terminal 1 - Riyadh",
    "serial_number": "EGS1-886431145-0001",
    "otp": "123456",
    "environment": "sandbox",
    "is_active": true
  }'
```

Expected output:

```
{
  "data": {
    "id": "<uuid>",
    "device_name": "POS Terminal 1 - Riyadh",
    "serial_number": "EGS1-886431145-0001",
    "environment": "sandbox",
    "status": "pending_registration",
    "is_active": true
  }
}
```


Problem 8.4: Submit an E-Invoice to ZATCA

Submit a simplified or standard e-invoice for clearance.

How to test:

```
curl -X POST http://localhost:3000/api/accounting/zatca-submissions \
  -H "Content-Type: application/json" \
  -H "Cookie: <your-session-cookie>" \
  -d '{
    "invoice_id": "<invoice-uuid>",
    "submission_type": "simplified",
    "invoice_hash": "NWM4NTJiNTUyZjQ5YWY2MDdmMjI=",
    "xml_content": "<Invoice>...</Invoice>"
  }'
```

Expected output:


```
{
  "data": {
    "id": "<uuid>",
    "invoice_id": "<invoice-uuid>",
    "submission_type": "simplified",
    "status": "pending",
    "submitted_at": "2026-07-23T..."
  }
}
```

Phase 9: Intercompany & Consolidation

Problem 9.1: Create an Intercompany Agreement

Define terms between related legal entities.

How to test:

```
curl -X POST http://localhost:3000/api/accounting/ic-agreements \
  -H "Content-Type: application/json" \
  -H "Cookie: <your-session-cookie>" \
  -d '{
    "name_ar": "اتفاقية خدمات مشتركة",
    "name_en": "Shared Services Agreement",
    "entity_a_id": "<legal-entity-1-id>",
    "entity_b_id": "<legal-entity-2-id>",
    "agreement_type": "service",
    "effective_date": "2026-01-01",
    "is_active": true
  }'
```

Expected output:

```
{
  "data": {
    "id": "<uuid>",
    "name_en": "Shared Services Agreement",
    "entity_a_id": "<entity-1>",
    "entity_b_id": "<entity-2>",
    "agreement_type": "service",
    "is_active": true
  }
}
```

Problem 9.2: Create a Consolidation Group

Group entities for consolidated financial reporting.

How to test:

```
curl -X POST http://localhost:3000/api/accounting/consolidation-groups \
  -H "Content-Type: application/json" \
  -H "Cookie: <your-session-cookie>" \
  -d '{
    "name_ar": "المجموعة الموحدة",
    "name_en": "Consolidated Group",
    "parent_entity_id": "<holding-entity-id>",
    "consolidation_currency": "SAR",
    "is_active": true
  }'
```

Expected output:

```
{
  "data": {
    "id": "<uuid>",
    "name_en": "Consolidated Group",
    "parent_entity_id": "<holding-entity-id>",
    "consolidation_currency": "SAR",
    "is_active": true
  }
}
```

Problem 9.3: Run a Consolidation

Execute consolidation to generate combined financial statements.

How to test:

```
curl -X POST http://localhost:3000/api/accounting/consolidation-runs \
  -H "Content-Type: application/json" \
  -H "Cookie: <your-session-cookie>" \
  -d '{
    "group_id": "<consolidation-group-id>",
    "period_start": "2026-01-01",
    "period_end": "2026-06-30",
    "run_type": "full"
  }'
```

Expected output:

```
{
  "data": {
    "id": "<uuid>",
    "group_id": "<consolidation-group-id>",
    "period_start": "2026-01-01",
    "period_end": "2026-06-30",
    "status": "draft",
    "run_type": "full"
  }
}
```

Problem 9.4: Create an Elimination Rule

Rules for eliminating intercompany balances during consolidation.

How to test:

```
curl -X POST http://localhost:3000/api/accounting/elimination-rules \
  -H "Content-Type: application/json" \
  -H "Cookie: <your-session-cookie>" \
  -d '{
    "name_ar": "استبعاد المبيعات البينية",
    "name_en": "Eliminate IC Revenue",
    "group_id": "<consolidation-group-id>",
    "rule_type": "revenue_expense",
    "debit_account_id": "<ic-revenue-account>",
    "credit_account_id": "<ic-expense-account>",
    "is_active": true
  }'
```

Expected output:

```
{
  "data": {
    "id": "<uuid>",
    "name_en": "Eliminate IC Revenue",
    "rule_type": "revenue_expense",
    "is_active": true
  }
}
```

Phase 10: Reporting, Imports & Dashboard

Problem 10.1: Create a Report Definition

Define a reusable report template.

How to test:

```
curl -X POST http://localhost:3000/api/accounting/report-definitions \
  -H "Content-Type: application/json" \
  -H "Cookie: <your-session-cookie>" \
  -d '{
    "name_ar": "تقرير المصروفات الشهرية",
    "name_en": "Monthly Expense Report",
    "report_type": "expense_analysis",
    "query_template": "SELECT account_id, SUM(debit) as total FROM accounting_journal_l
    "parameters": {"period": "monthly", "group_by": "cost_center"},
    "output_format": "table",
    "is_active": true
  }'
```

Expected output:

```
{
  "data": {
    "id": "<uuid>",
    "name_en": "Monthly Expense Report",
    "report_type": "expense_analysis",
    "output_format": "table",
    "is_active": true
  }
}
```

Problem 10.2: Schedule a Report

Set up automatic report generation.

How to test:

```
curl -X POST http://localhost:3000/api/accounting/report-schedules \
  -H "Content-Type: application/json" \
  -H "Cookie: <your-session-cookie>" \
  -d '{
    "report_definition_id": "<report-def-id-from-10.1>",
    "schedule_name": "Weekly Expense Summary",
    "cron_expression": "0 8 * * 1",
    "recipients": ["finance@company.sa"],
    "is_active": true
  }'
```

Expected output:

```
{
  "data": {
    "id": "<uuid>",
    "report_definition_id": "<report-def-id>",
    "schedule_name": "Weekly Expense Summary",
    "cron_expression": "0 8 * * 1",
    "is_active": true
  }
}
```

Problem 10.3: Run a Report Manually

Execute a report on-demand.

How to test:

```
curl -X POST http://localhost:3000/api/accounting/report-runs \
  -H "Content-Type: application/json" \
  -H "Cookie: <your-session-cookie>" \
  -d '{
    "report_definition_id": "<report-def-id>",
    "parameters": {"date_from": "2026-07-01", "date_to": "2026-07-31"},
    "triggered_by": "manual"
  }'
```

Expected output:

```
{
  "data": {
    "id": "<uuid>",
    "report_definition_id": "<report-def-id>",
    "status": "pending",
    "triggered_by": "manual",
    "started_at": "2026-07-23T..."
  }
}
```

Problem 10.4: Create an Import Definition

Define how external data should be imported.

How to test:

```
curl -X POST http://localhost:3000/api/accounting/import-definitions \
  -H "Content-Type: application/json" \
  -H "Cookie: <your-session-cookie>" \
  -d '{
    "name_ar": "استيراد كشف البنك",
    "name_en": "Bank Statement Import",
    "source_type": "csv",
    "target_table": "bank_statement_lines",
    "column_mapping": {
      "date": "transaction_date",
      "description": "narrative",
      "amount": "amount",
      "reference": "reference_no"
    },
    "is_active": true
  }'
```

Expected output:

```
{
  "data": {
    "id": "<uuid>",
    "name_en": "Bank Statement Import",
    "source_type": "csv",
    "target_table": "bank_statement_lines",
    "is_active": true
  }
}
```

Problem 10.5: Create a Dashboard Widget

Configure dashboard components for quick views.

How to test:

```
curl -X POST http://localhost:3000/api/accounting/dashboard-widgets \
  -H "Content-Type: application/json" \
  -H "Cookie: <your-session-cookie>" \
  -d '{
    "name_ar": "ملخص النقدية",
    "name_en": "Cash Summary",
    "widget_type": "kpi_card",
    "data_source": "cash_positions",
    "config": {"metric": "total_balance", "currency": "SAR"},
    "position": 1,
    "is_active": true
  }'
```

Expected output:

```
{
  "data": {
    "id": "<uuid>",
    "name_en": "Cash Summary",
    "widget_type": "kpi_card",
    "position": 1,
    "is_active": true
  }
}
```


Phase 11: Reports & Financial Statements

Problem 11.1: Generate Trial Balance

Verify debits equal credits across all accounts.

How to test:

```
curl "http://localhost:3000/api/accounting/reports/trial-balance?as_of=2026-07-31" \
  -H "Cookie: <your-session-cookie>"
```

Expected output:

```
{
  "data": {
    "as_of": "2026-07-31",
    "accounts": [
      {
        "account_id": "<uuid>",
        "code": "1100",
        "name_ar": "النقد وما في حكمه",
        "name_en": "Cash and Cash Equivalents",
        "debit_balance": 45000,
        "credit_balance": 0
      },
      {
        "account_id": "<uuid>",
        "code": "5100",
        "name_ar": "مصروفات الإيجار",
        "name_en": "Rent Expense",
        "debit_balance": 5000,
        "credit_balance": 0
      }
    ],
    "total_debit": 50000,
    "total_credit": 50000
  }
}
```


The totals MUST balance (total_debit = total_credit). If they don't, there's a posting error.

Problem 11.2: Generate Balance Sheet

Financial position at a point in time.

How to test:

```
curl "http://localhost:3000/api/accounting/reports/balance-sheet?as_of=2026-07-31" \
  -H "Cookie: <your-session-cookie>"
```

Expected output:

```
{
  "data": {
    "as_of": "2026-07-31",
    "assets": {
      "current_assets": [...],
      "non_current_assets": [...],
      "total": 500000
    },
    "liabilities": {
      "current_liabilities": [...],
      "non_current_liabilities": [...],
      "total": 200000
    },
    "equity": {
      "items": [...],
      "total": 300000
    }
  }
}
```


Assets MUST equal Liabilities + Equity. If not, the balance sheet is incorrect.

Problem 11.3: Generate Profit & Loss Statement

Revenue and expenses over a period.

How to test:

```
curl "http://localhost:3000/api/accounting/reports/profit-loss?date_from=2026-07-01&dat
  -H "Cookie: <your-session-cookie>"
```

Expected output:

```
{
  "data": {
    "period": {"from": "2026-07-01", "to": "2026-07-31"},
    "revenue": {
      "items": [...],
      "total": 200000
    },
    "expenses": {
      "items": [...],
      "total": 150000
    },
    "net_income": 50000
  }
}
```


Problem 11.4: Generate General Ledger

Detailed transaction history per account.

How to test:

```
curl "http://localhost:3000/api/accounting/reports/general-ledger?account_id=<account-u
  -H "Cookie: <your-session-cookie>"
```

Expected output:

```
{
  "data": {
    "account": {"id": "...", "code": "1100", "name_en": "Cash"},
    "opening_balance": 50000,
    "transactions": [
      {
        "date": "2026-07-01",
        "entry_no": "JV-000001",
        "description": "Office rent payment",
        "debit": 0,
        "credit": 5000,
        "running_balance": 45000
      }
    ],
    "closing_balance": 45000
  }
}
```


Error Scenarios to Test

Error 1: Unbalanced Journal Entry

Debits must equal credits.

How to test:

```
curl -X POST http://localhost:3000/api/accounting/journals \
  -H "Content-Type: application/json" \
  -H "Cookie: <your-session-cookie>" \
  -d '{
    "journal_date": "2026-07-01",
    "description": "Bad entry",
    "lines": [
      {"account_id": "<id>", "debit": 5000, "credit": 0},
      {"account_id": "<id>", "debit": 0, "credit": 4000}
    ]
  }'
```

Expected output:

```
{
  "message": "Total debits (5000) must equal total credits (4000)."
}
```

Status code: `400`

Error 2: Post to a Closed Period

Cannot post to a period that has been closed.

How to test:

Close a period first, then try to post a journal entry dated within that period.

Expected output:

```
{
  "message": "Cannot post to closed period."
}
```

Status code: `409`

Error 3: Unauthorized Access (viewer trying to post)

Users with `finance_viewer` role cannot modify data.

How to test:

Log in as a user with `finance_viewer` role and attempt to create a journal entry.

Expected output:

```
{
  "message": "Insufficient accounting permissions."
}
```

Status code: `403`

Error 4: Duplicate Idempotency Key

Sending the same command twice with the same idempotency key.

How to test:

```
curl -X POST http://localhost:3000/api/accounting/journals \
  -H "Content-Type: application/json" \
  -H "Cookie: <your-session-cookie>" \
  -H "Idempotency-Key: 550e8400-e29b-41d4-a716-446655440000" \
  -d '{...valid journal data...}'

# Send the exact same request again
curl -X POST http://localhost:3000/api/accounting/journals \
  -H "Content-Type: application/json" \
  -H "Cookie: <your-session-cookie>" \
  -H "Idempotency-Key: 550e8400-e29b-41d4-a716-446655440000" \
  -d '{...valid journal data...}'
```

Expected output (second request):

```
{
  "message": "Command already processed."
}
```

Status code: `409`

Testing Sequence (Recommended Order)

Follow this order to build up test data progressively:

1. **Initialize** (Problem 1.1) - Creates default accounts and settings
2. **Create entity** (Problem 1.2) - Required for multi-entity scenarios
3. **Create fiscal calendar** (Problem 1.4) - Required for period validation

4. **Set up COA accounts** (Problem 1.5) - Verify defaults created
5. **Create cost center** (Problem 2.1) - For dimension-based reporting
6. **Create journal entry** (Problem 3.1) - Core transaction
7. **Post journal** (Problem 3.2) - Finalize transaction
8. **Check trial balance** (Problem 11.1) - Verify posting
9. **Create bank account** (Problem 4.1) - For treasury
10. **Register asset** (Problem 5.2) - For fixed assets
11. **Create party** (Problem 6.1) - For AR/AP
12. **Create budget** (Problem 7.1) - For budget control
13. **Set up tax codes** (Problem 8.1) - For VAT compliance
14. **Run reports** (Problems 11.1-11.4) - Verify everything ties out

Quick Validation Checklist

Check	How to Verify	Pass Criteria
Trial balance balances	GET `/reports/trial-balance`	`total_debit == total_credit`
Balance sheet equation	GET `/reports/balance-sheet`	`assets == liabilities + equity`
Journal entry integrity	POST `/journals` with unequal lines	Returns 400 error
RLS enforcement	Query as different org user	Returns empty array
Idempotency	POST same command_id twice	Second returns 409
Role enforcement	POST as viewer	Returns 403
Period control	Post to closed period	Returns 409
Currency consistency	All amounts in SAR (default)	No conversion errors
Arabic names stored	GET any entity	`name_ar` field populated

Check	How to Verify	Pass Criteria
Audit trail	Check `accounting_commands` table	Every mutation logged